

OBJEKTORIENTERET ANALYSE, DESIGN OG IMPLEMENTATION

1: Introduction to OO

Jan H. Mikkelsen
Cyber og Computerteknologi
2. semester – KBH – F26

AGENDA

1

- Introduction to the course (and a bit of public service information)
- Introduction to Object Oriented Programming, as opposed to imperativ (procedural) programming
- The Java background
 - The role of Java
 - What is Java actually
- Getting started .. what is needed to start with Java programming
- Example of a simple Java implementation (Hello World)
- Tips & tricks
- Assignments

“Quite a few” slides but many
are related to IDE details
and I will go very quickly through those

PUBLIC SERVICE INFORMATION

2



Jan H. Mikkelsen

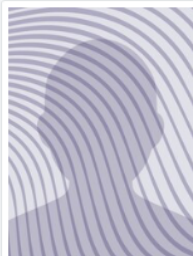
Lektor
Institut for Elektroniske Systemer
Det Tekniske Fakultet for IT og Design
Antennas, Propagation and Millimetre-Wave Systems
AI RF-sensors
<https://orcid.org/0000-0003-2366-8831>



Vis Scopus-profil

Telefon
+4599408665
E-mail
hmi@es.aau.dk

Selma Lagerløfs Vej 312, 3-311
9220 Aalborg Øst
Danmark



Severen Fernandes

Videnskabelig assistent
Institut for Elektroniske Systemer
Det Tekniske Fakultet for IT og Design
Communication, Media and Information technologies
Cyber Security Group
<https://orcid.org/0009-0009-0618-9629>

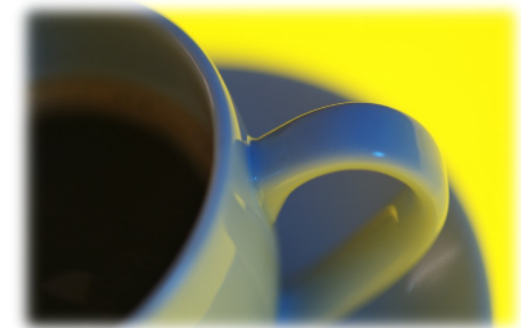
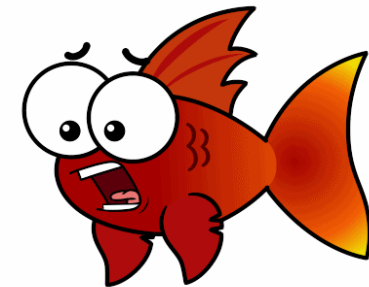
Telefon
+4599409357
E-mail
sef@es.aau.dk

Frederikskaj 12
2450 København SV
Danmark

PUBLIC SERVICE INFORMATION

3

- I try to include a few smallish breaks along the way
 - The tiny fish tells us to stay in the room while we roll our eyes a bit and maybe annoy the person sitting next to us,
 - The nice coffee cup tells us that it is time for a proper break (10-15 min.)
- PLEASE do interrupt me if you have questions along the way
- As you know there are (almost) no stupid questions
- Everything is easy if you know how to “do it”
- If not, then everything is f***ing difficult



COURSE OVERVIEW

- Modulets indhold, forløb og pædagogik
 - At bibringe studerende forståelse af og erfaring med objektorienteret softwareudvikling, herunder praktisk erfaring med programmering i objektorienterede sprog, f.eks. Java eller C++, via omfattende kode-eksempler
- Læringsmål
 - Viden
 - objektorienteret analyse af problemområdet
 - objektorienteret analyse af anvendelsesområdet, herunder funktionelle krav og brugsmønstre
 - objektorienteret design, herunder forskellige principper for design
 - objektorienteret implementering, herunder udviklingsværktøjer, programmeringssprog og automatisk kodegenerering
 - centrale begreber inden for objektorientering, herunder bl.a. klasser, objekter, nedarving, interfaces, exception handling m.m.

COURSE OVERVIEW

5

- Færdigheder
 - kan på et systematisk grundlag udvikle objektorienteret software
 - kan udnytte de væsentligste funktionaliteter i det valgte programmeringssprog
 - kan dokumentere software på en måde, som sætter andre fagpersoner i stand til at vedligeholde og videreudvikle softwaren
 - kan udvikle komplekse softwaresystemer ved brug af objektorienterede sprog og principper
- Kompetencer
 - kan løse relevante problemstillinger ved brug af objektorienterede principper
 - kan argumentere for valgte løsninger, herunder redegøre for begrænsninger

COURSE OVERVIEW

- Topics covered (not necessarily in this order though)
 1. Basic intro
 2. UML and use-case
 3. Classes and objects
 4. Class relation and class diagrams
 5. Data structures and types
 6. Abstraction, interfaces and polymorphism
 7. Lists, iterators and operators (e.g. compare objects)
 8. Threads, concurrency and synchronization
 9. Dynamic interactions and sequences
 10. IO, Communications and events
 11. Exceptions and error handling
 12. Testing

AGENDA

7

- Introduction to the course (and a bit of public service information)
- Introduction to Object Oriented Programming, as opposed to imperativ (procedural) programming
- The Java background
 - The role of Java
 - What is Java actually
- Getting started .. what is needed to start with Java programming
- Example of a simple Java implementation (Hello World)
- Tips & tricks
- Assignments

OBJECT ORIENTED PROGRAMMING

- What is object oriented programming (OOP)?
- If uncle-Al is consulted, we get the following suggestion

“OOP is a programming paradigm, where software is structured around objects – small units containing both data and functionality. OOP is all about modelling the real-world using classes and objects”

- Classes
 - A class is a template or recipe that describes what data (attributes) and functions (methods) an object has
- Object
 - An object is a concrete instance of a class. If the class is the receipt then the object is the dish itself

OBJECT ORIENTED PROGRAMMING

9

- Where OOP organises programs into objects, and focuses on the objects and how they interact, then imperative programming is focused on describing the order of instructions ... how a task is to be implemented
- OOP bundles data and functionality into one unit – the object
- OOP operates on "models of reality", e.g. a 'Car' – what is a car and how is it possible to interact with a car
- OOP is an organization structure but underneath the hood, OOP is still imperative

IP example

```
1 int x = 5;  
2 x = x + 1;  
3 printf("%d", x);
```

OOP example

```
1 class Car {  
2   int speed;  
3  
4   void accelerate() {  
5     speed += 10;  
6   }  
7 }
```

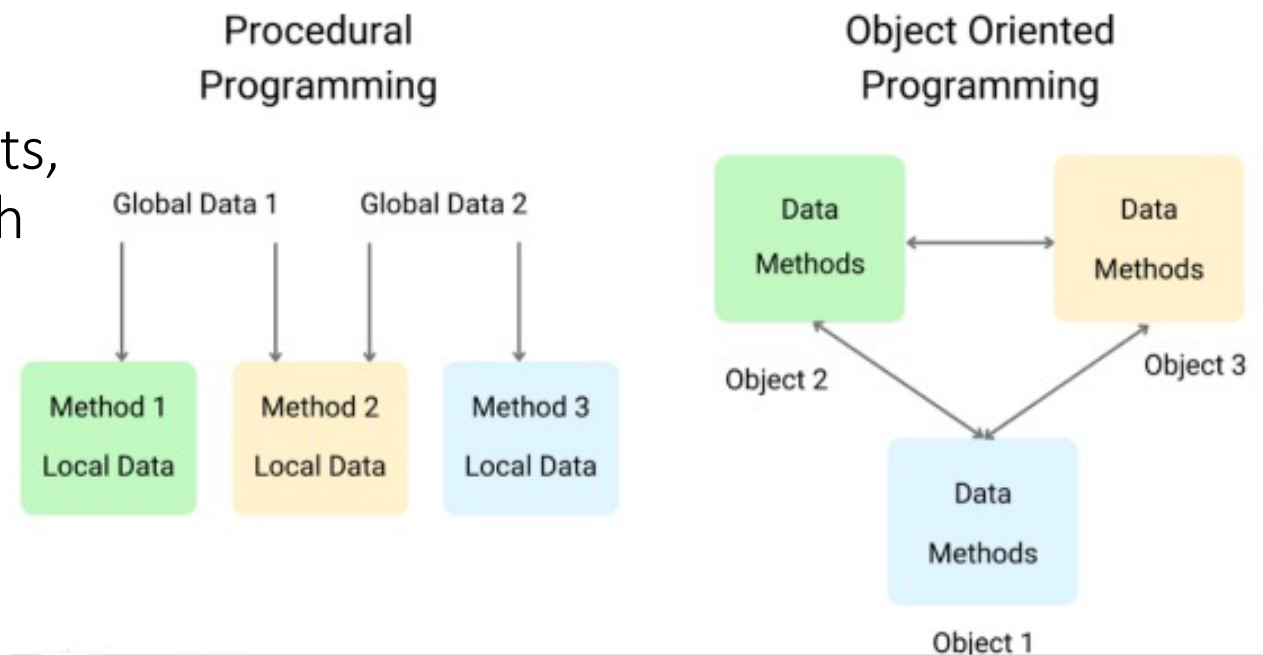
Object-oriented



OBJECT ORIENTED PROGRAMMING

10

- In the real-world, everything around us can be perceived as an object (three, car, phone, etc.)
- Objects are defined by attributes and behaviors
- OOP: A programming paradigm that models software as a set of objects, which are defined through blueprints (usually called classes)



OBJECT ORIENTED PROGRAMMING

11

- In summary

Aspect	Imperative Programming	Object-Oriented Programming (OOP)
1. Focus	How to perform tasks (step-by-step instructions)	What objects exist and how they interact
2. Code organization	Functions and procedures	Classes and objects
3. Data handling	Data and behavior are separate	Data and behavior are bundled together in objects
4. Reuse	Reuse through functions and modules	Reuse through inheritance, interfaces, polymorphism
5. Best suited for	Small, simple, procedural programs	Large, complex, modular systems

OBJECT ORIENTED PROGRAMMING

12

- OO did not appear out of the blue and people have manually been implementing OO-ish methodologies

C-example using struct

- This example is essentially:
 - a **class** (Point)
 - a **constructor** (initializing the struct)
 - **methods** (move, print)
 - passing this explicitly (Point* p)
- The programmer is in fact doing OO manually (unstructured!)
- If you analyse ordinary, practical, C-code many examples are seen

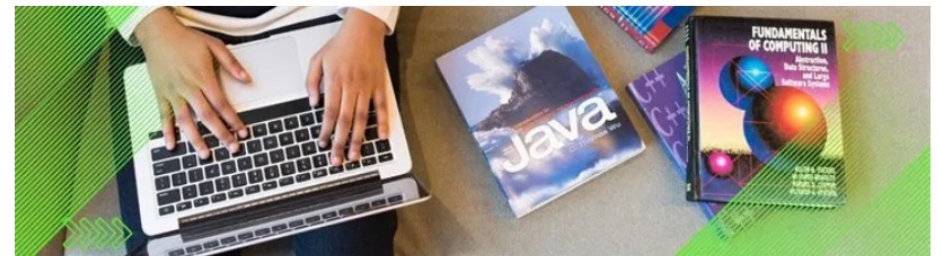
```
1  typedef struct {
2      int x;
3      int y;
4  } Point;
5
6  void Point_move(Point* p, int dx, int dy) {
7      p->x += dx;
8      p->y += dy;
9  }
10
11 void Point_print(const Point* p) {
12     printf("(%d, %d)\n", p->x, p->y);
13 }
```

OBJECT ORIENTED PROGRAMMING

13

- A lot of programming languages support the OOP approach – some of the most wellknown are Java, C++, C# and Python (with OOP-addon)
- We have chosen to use Java in this course
 - As a teaching tool for OOP, Java is well suited as it requires an OO approach
 - Other languages, such as C#, supports OO, but it is still possible to implement code in non-OO fashion
 - Java is widely used and it sees a lot of use in industry, where as an example the Android mobile system depends heavily on Java
 - And not least then it is a mature language

Java



Java remains one of the most widely adopted OOP languages across enterprise environments. Released by Sun Microsystems in 1995, it popularized the "write once, run anywhere" philosophy through its Java Virtual Machine.

The language enforces strong typing and requires everything to exist within classes. There are no standalone functions floating around. This strict structure makes Java code predictable and maintainable across large teams.

Key uses include **Android mobile applications**, enterprise backend systems, financial services platforms, and large-scale web applications. Banks, insurance companies, and government agencies frequently choose Java for mission-critical systems.

OBJECT ORIENTED PROGRAMMING

14

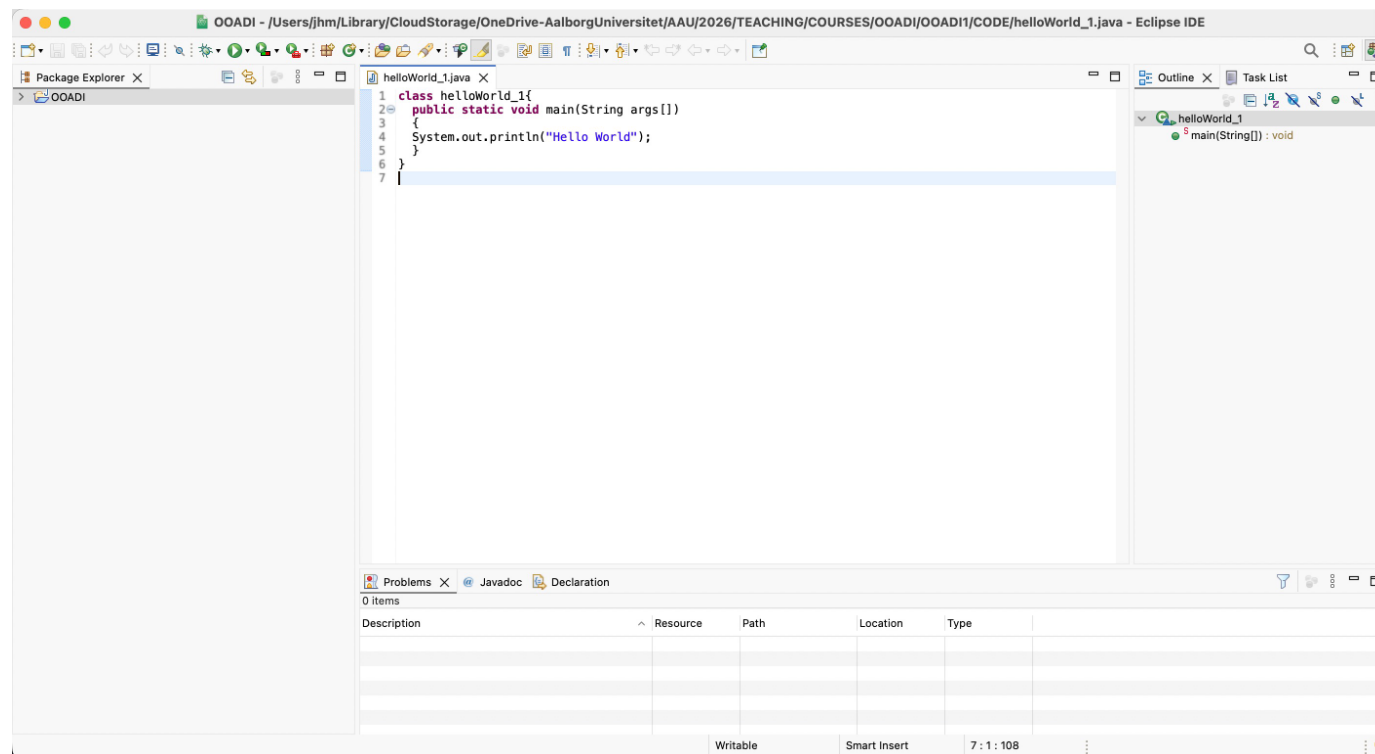
- Looking at the editor part of things, then a simple text editor will get us a very long way
- Many editors, such as Atom, supports different syntaxes and can help with formatting (colors) to improve overview and readability
- The implemented code can then be compiled and executed through the use of a command line prompt
- The disadvantage – as complexity grows the need for debugging may arise and this is where the IDE comes into the picture
- On the IDE-front, many options are available, all with their unique features and quirks

```
1  class helloWorld_1{
2      public static void main(String args[])
3      {
4          System.out.println("Hello World");
5      }
6  }
```

OBJECT ORIENTED PROGRAMMING

15

- In this course we will try to stick to Eclipse, as this tool contains all that is needed and it supports both Windows, Linux and MacOS



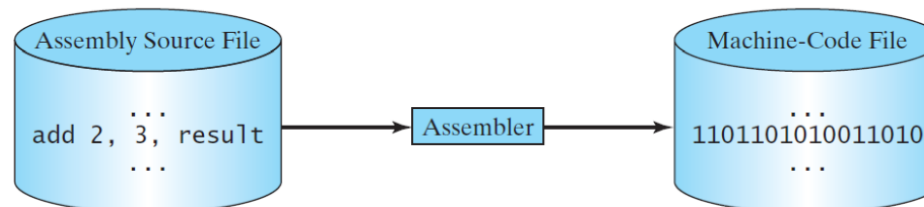
OBJECT ORIENTED PROGRAMMING

16

- A computer understands only zeros and ones and as a first layer of abstraction assembler programming was invented
- Despite being developed to make programming easy, it is nonetheless a language closely tied to the actual hardware and consists of a set of primitive instructions – “efficient” but hopelessly difficult to read
- Values are often given in binary or hexadecimal form
- To convert assembly language programs into machine code an assembler program is needed

AMS example

```
1  MOV AX, 5
2  MOV BX, 3
3  ADD AX, BX
```

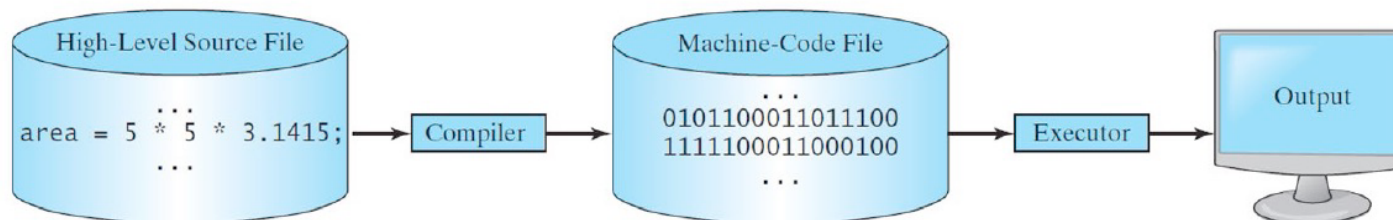


OBJECT ORIENTED PROGRAMMING

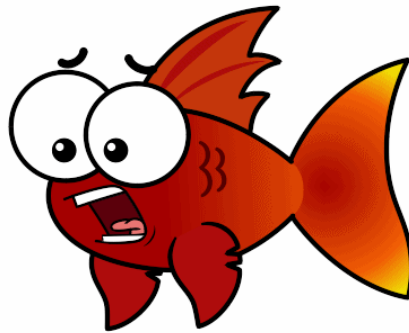
17

- High-level programming languages are a lot easier to learn og they provide greatly improved readabillity
- A program written in a high-level language, such as Java, is called source code (well .. so is assempley code) and here here operations can be described in a way where the actual code line is selfexplanatory
- The road to zeros and ones is now even longer and the need for an efficient compiler is obvious

```
1 radius = 5;  
2 area = radius * radius * 3.1415;
```



BREAK – PAUSE(FISK)



AGENDA

19

- Introduction to the course (and a bit of public service information)
- Introduction to Object Oriented Programming, as opposed to imperative (procedural) programming
- The Java background
 - The role of Java
 - What is Java actually
- Getting started .. what is needed to start with Java programming
- Example of a simple Java implementation (Hello World)
- Tips & tricks
- Assignments

JAVA – WHY?

20

- Java was “invented” by J. Gosling, M. Sheridan and P. Naughton back in 1991
- Sun Microsystems launched the first public version of Java (Java 1.0) back in 1996
- The driving focus was the Write Once, Run Anywhere (WORA) principle and (likely) to ensure recognizability they adopted a C/C++ syntax style
- Throughout its life, five goals (or objectives) have continued to drive the development of Java
 - Simple, object oriented og recognizable
 - Robust and secure
 - Architecture neutral and portable
 - High performance
 - Interpreted, treaded and dynamic



JAVA – WHAT?

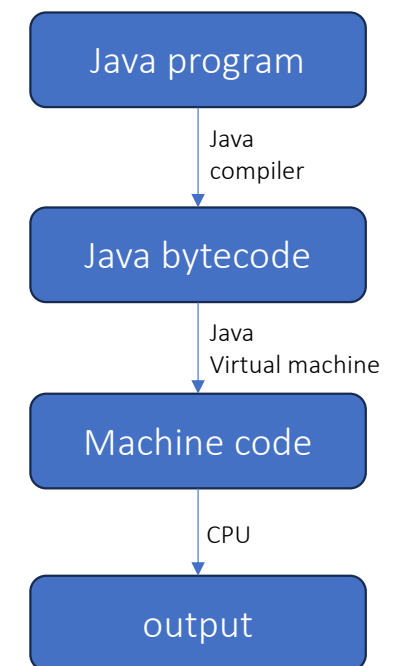
- Java is clearly a high-level programming language
- It is implemented in a way where implementation dependencies are as few as possible
- It is – as mentioned – a general-purpose language ... a WORA language
- Java code is compiled into byte code and executed in a Java virtual machine – for that reason, a Java Runtime Environment (JRE) must be installed as minimum
- In addition, we have the Java Development Kit (JDK), which contains a development layer as well as the JRE
- Java is used for both server and client software, while javascript is used for frontend browser code
- It is important to be aware of this distinction

Java ≠ JavaScript

JAVA – WHAT?

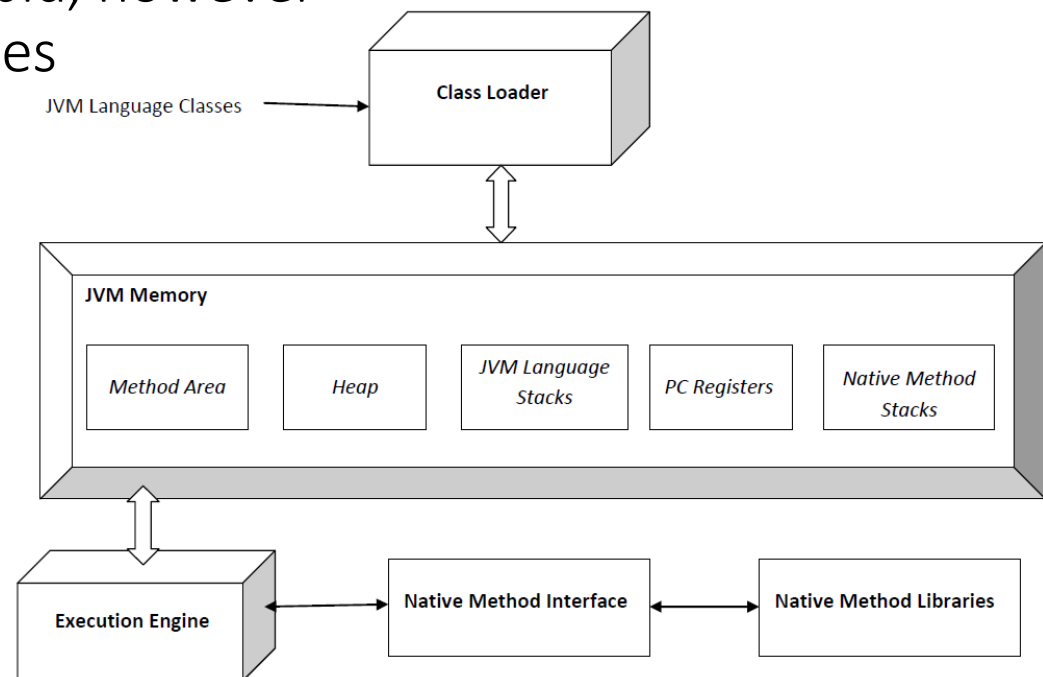
22

- As mentioned, to run a Java program we must first have it compiled into byte code
- Byte code
 - This is much like native machine code but CPU independent
 - However, the JVM is NOT independent
 - Some microcontrollers can natively execute byte code directly
- The ambition for high performance suffered initially but around 1997/1998, just-in-time compiling was introduced and so were much better options for code analysis
- Today, implementation strategies for algorithms are far more important. No significant performance differences can be related to the choice of programming language



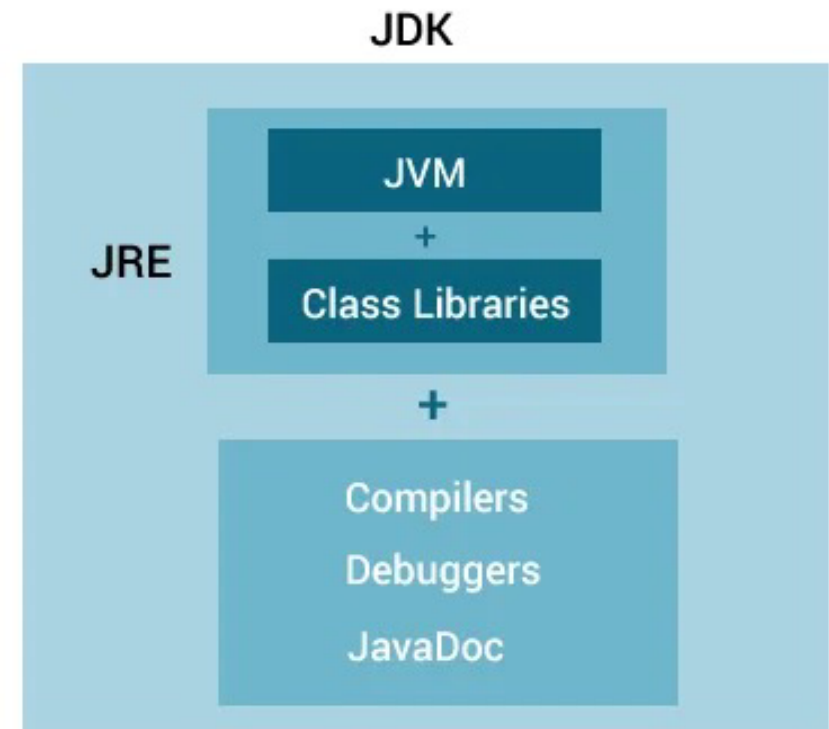
JAVA – WHAT?

- All code depends on the specific CPU used, the memory, the OS, hardware, drivers etc.
- The use of a VM may seem stupid, however it provides for several advantages
 - Separation of byte code. Here machine code and application code optimization is separated from each other
 - Much simpler compilation process
 - The Java compiler can focus on the Java environment



JAVA – WHAT?

- This takes us back to the JRE and JDK elements again as they define the combined Java environment
 - Java Runtime Environment (JRE)
 - Contains all packages/libraries needed to execute the Java byte code
 - Java Development Kit (JDK)
 - Contains everything related to development of Java code. This includes compiler, javadoc, Jar and debugger ... everything!
- Dividing code into those two segments allows us to focus download and use local storage to the purpose we have
- The JDK does not always include JRE!!!

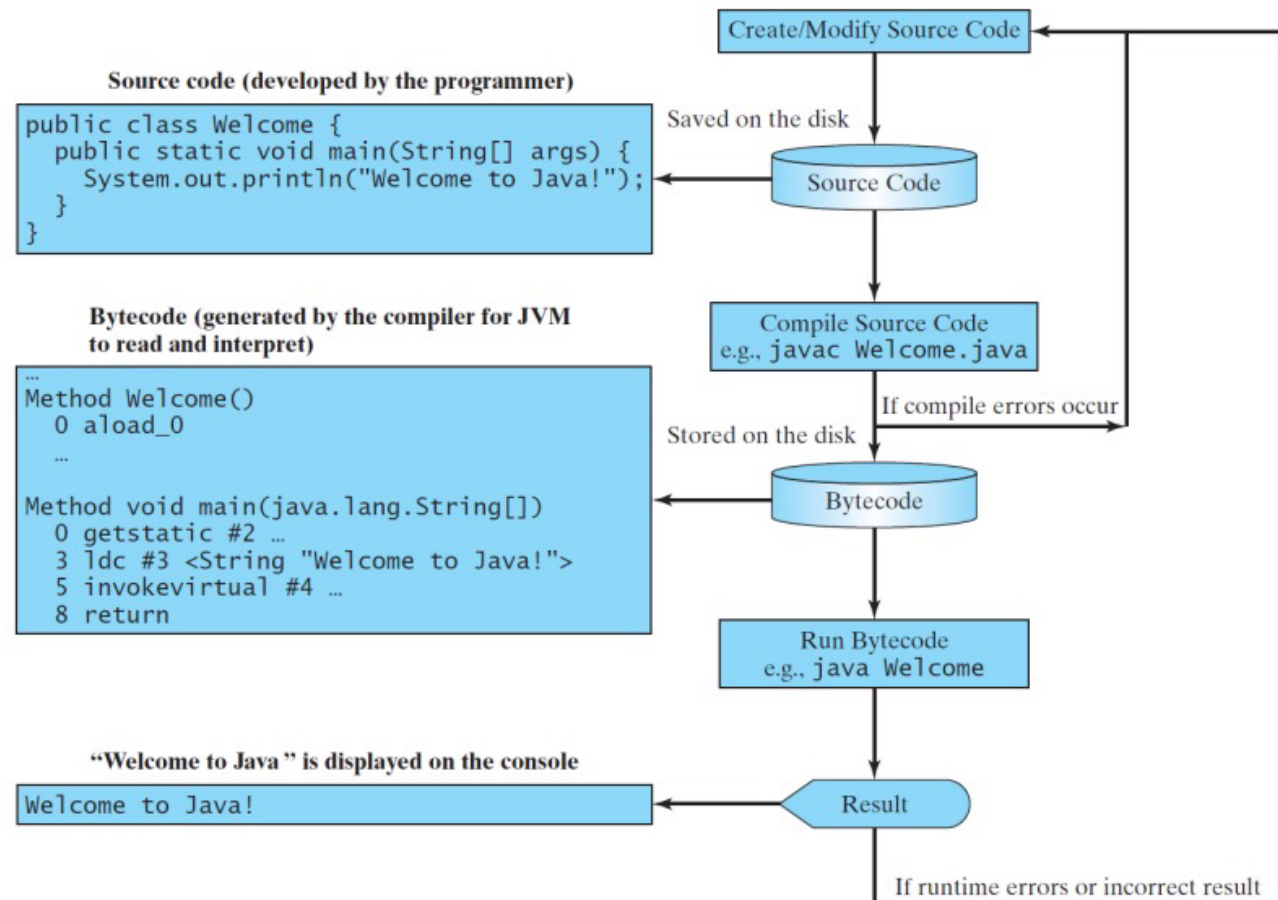


Ref: <https://www.programiz.com/java-programming/jvm-jre-jdk>

JAVA – WHAT?

25

- The programming flow, when writing in Java, follows the procedure outlined here to the right



AGENDA

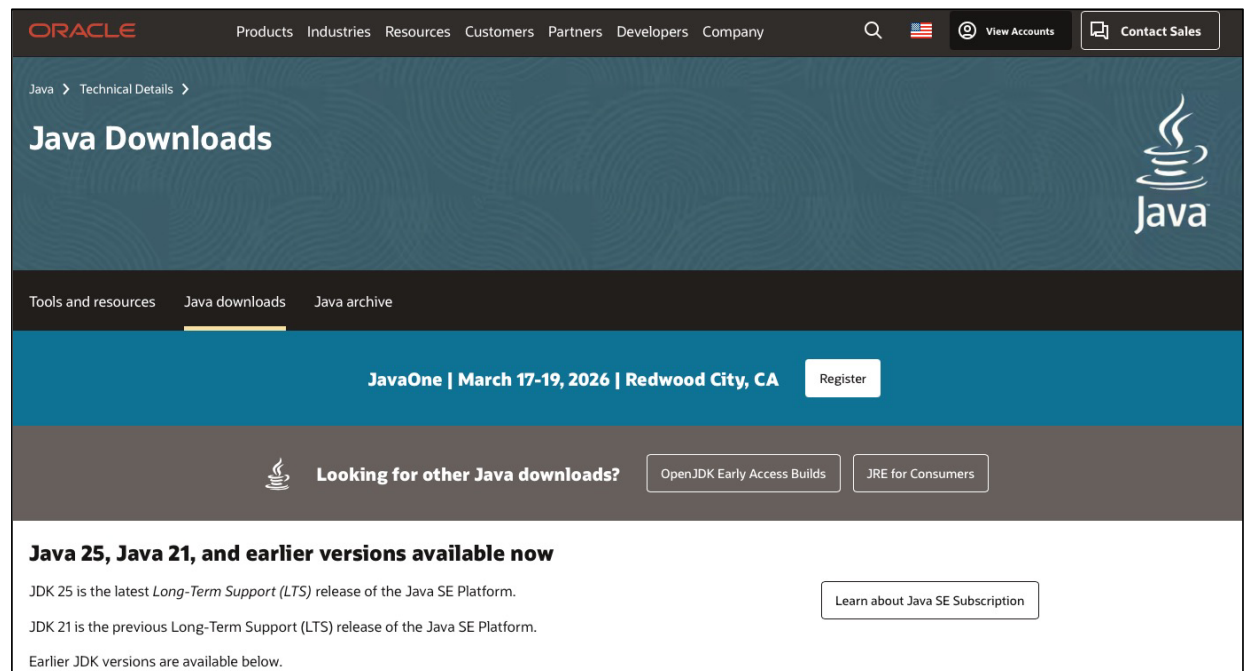
26

- Introduction to the course (and a bit of public service information)
- Introduction to Object Oriented Programming, as opposed to imperativ (procedural) programming
- The Java background
 - The role of Java
 - What is Java actually
- Getting started .. what is needed to start with Java programming
- Example of a simple Java implementation (Hello World)
- Tips & tricks
- Assignments

GETTING STARTED

27

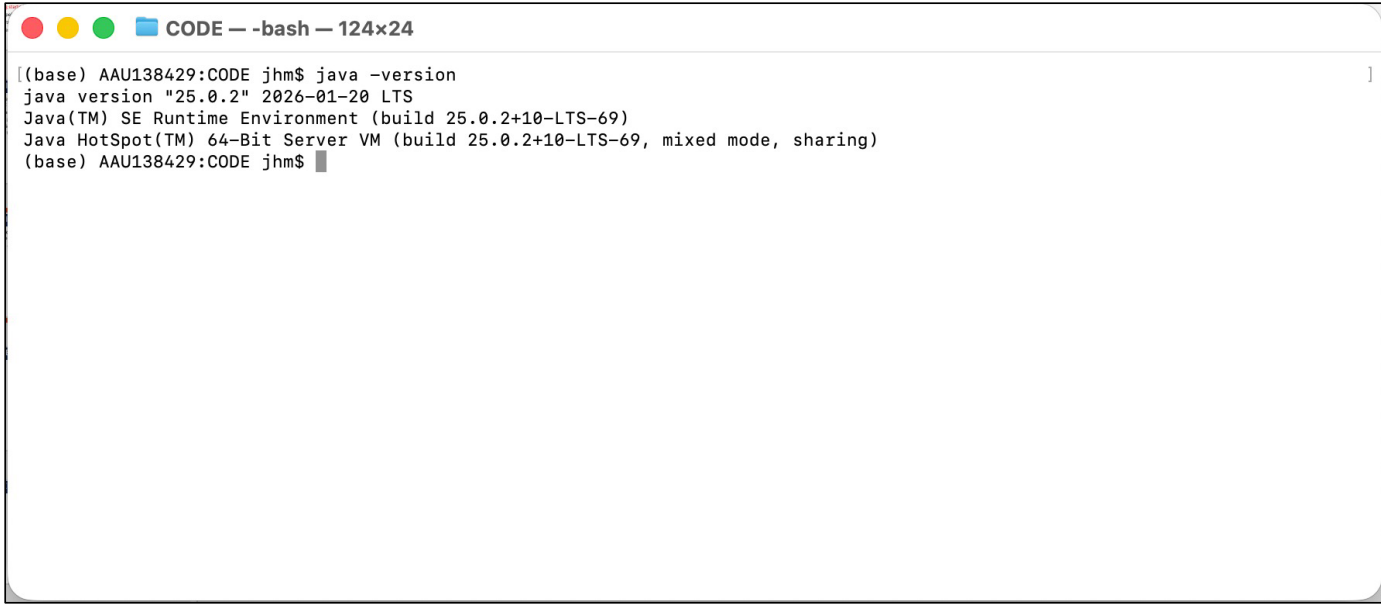
- The first step is to download Java, which means the JDK environment
<https://www.oracle.com/java/technologies/downloads/>
- Make sure to get the right version ... whether you need a 32- or a 64-bit version
- At the same time there are a gazillion different versions – the latest (January 2026) being Java 25



GETTING STARTED

28

- With the JDK installed, we can open a console/terminal and check the version of Java – here on MacOS
- Once this is done all we need is a simple text editor and we can get on with coding
- It is possible to both compile and execute Java code from a command line
- Nice for small pieces of code!



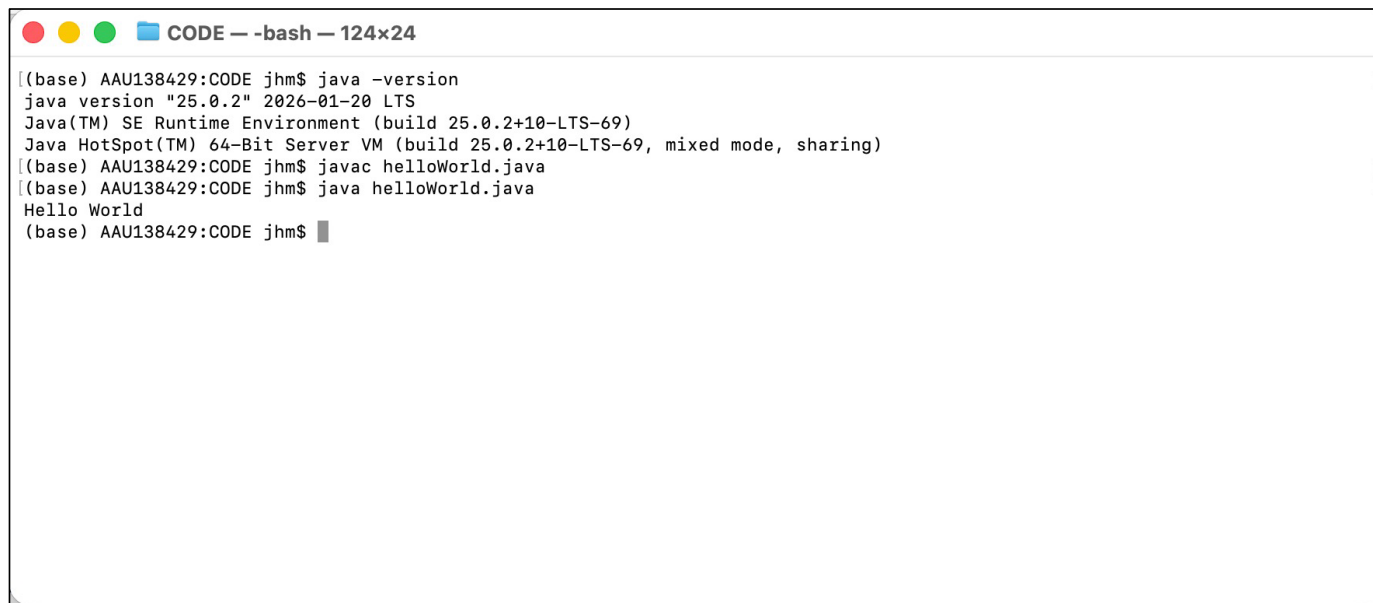
```
CODE — -bash — 124x24

[(base) AAU138429:CODE jhm$ java -version
java version "25.0.2" 2026-01-20 LTS
Java(TM) SE Runtime Environment (build 25.0.2+10-LTS-69)
Java HotSpot(TM) 64-Bit Server VM (build 25.0.2+10-LTS-69, mixed mode, sharing)
(base) AAU138429:CODE jhm$
```

GETTING STARTED

29

- From the command line the process is quite straight forward
- First, we use 'javac' to compile and after that we use 'java' to execute the code

A terminal window titled "CODE — -bash — 124x24" with standard macOS window controls (red, yellow, green buttons). The terminal shows a sequence of commands and their outputs. The first command is 'java -version', which outputs the Java version and runtime environment details. The second command is 'javac HelloWorld.java', which compiles the file. The third command is 'java HelloWorld.java', which executes the program and outputs 'Hello World'. The prompt '(base) AAU138429:CODE jhm\$' is visible at the end of each line.

```
[(base) AAU138429:CODE jhm$ java -version  
java version "25.0.2" 2026-01-20 LTS  
Java(TM) SE Runtime Environment (build 25.0.2+10-LTS-69)  
Java HotSpot(TM) 64-Bit Server VM (build 25.0.2+10-LTS-69, mixed mode, sharing)  
[(base) AAU138429:CODE jhm$ javac HelloWorld.java  
[(base) AAU138429:CODE jhm$ java HelloWorld.java  
Hello World  
(base) AAU138429:CODE jhm$
```

JAVA SYNTAX – IN GENERAL

- As mentioned, Java has adopted the C/C++ syntax and for that reason Java code resembles a lot of other code
- However, as an all-important detail, EVERYTHING is constructed using classes in Java

```
1 class HelloWorld {  
2 }
```

- This is followed by methods – and for now we will focus only on one specific method, namely main()

```
1 class HelloWorld {  
2     public static void main(String[] args) {  
3     }  
4 }
```

JAVA SYNTAX – IN GENERAL

31

- Every executable Java program consists of a public class
 - Contains a method named main, that contains the statements to be executed
 - A public class must be placed in file with a name of the form ClassName.java, e.g. class HelloWorld is stored in HelloWorld.java
 - Always start with a capital letter ... camel syntax generally promoted

```
public class Name {  
    public static void main(String[] args) {  
        statement;  
        statement;  
        ...  
        statement;  
    }  
}
```

class: a program

method: a named group
of statements

statement: a command to be executed

JAVA SYNTAX – IN GENERAL

32

- Scoping of code is handled via { }
- Each code line is ended by ;
- Printing to screen/console

```
1 System.out.println("Hello World");
```

- For-loops

```
1 for (int i=1; i<10; i++) {  
2     <do some stuff>;  
3 }
```

- While-loops

```
1 while (something) {  
2     <do some stuff>;  
3 }
```

In addition, we also have

```
1 if (something) {  
2     <do some stuff>;  
3 } else {  
4     <do other stuff>;  
5 }
```

```
1 switch (variable) {  
2     case A: ... break;  
3     case B: ... break;  
4     default: ... break;  
5 }
```

```
1 try {  
2     <something>;  
3 } catch (Exception e) {  
4     <something>;  
5 }
```

JAVA SYNTAX – METHODS AND THEIR USE

33

- Functions are called methods in Java and in OO generally
- Functions and methods appear very similar but conceptually they are very different

Function – a piece of code designed to do a task

Method – a function belonging to a class or an object

- Methods have (can have) input, output and functionality and the definition is

```
1  int someFunction(int z, int y) {  
2    ... kode der benytter x og y ... fx sqrt(x^2 + y^2)  
3    return result; // skal her være int  
4  }
```

JAVA SYNTAX – METHODS AND THEIR USE

34

- If we wish the method to output something, it is important that we ***return*** something of that specific data type (it can be anything)
- When we wish to make use of a method, we simply just use the name of the method

```
1  ...  
2  <et eller andet kode>;  
3  someFunction(3,4);  
4  <endnu mere kode>;  
5  ...
```

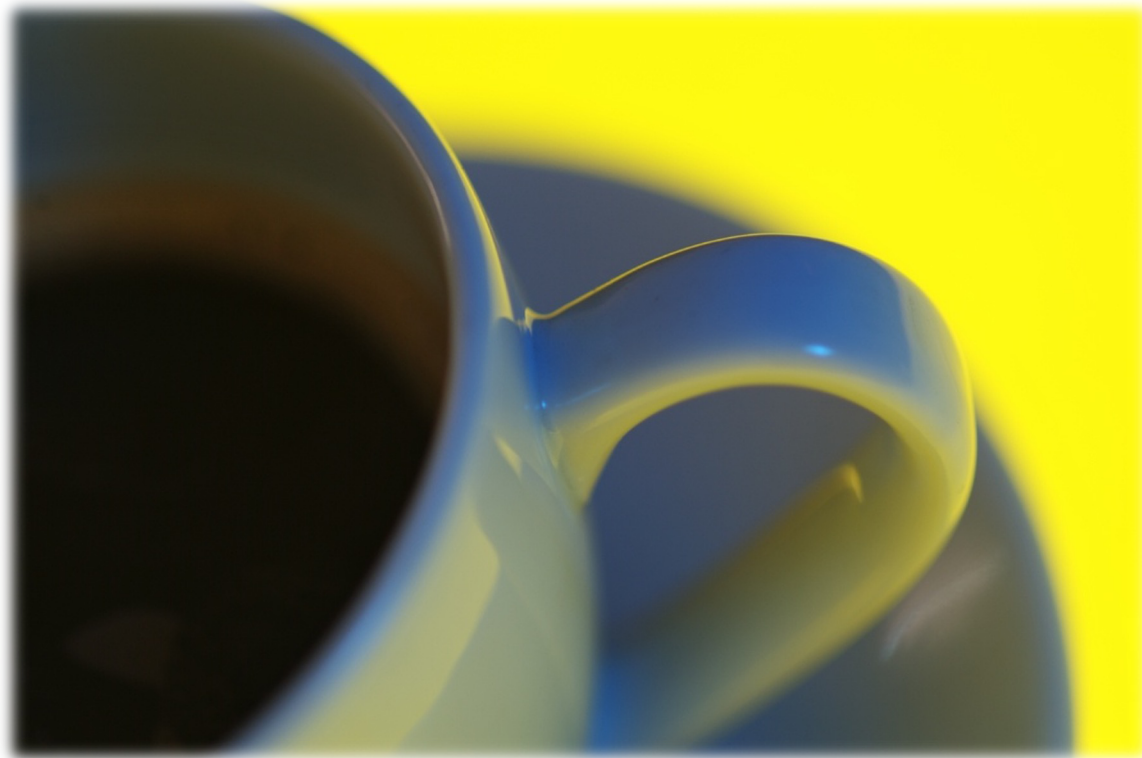
JAVA SYNTAX – BASIC DATA TYPES

35

- Here, everything is as we know from imperative programming

<i>Name</i>	<i>Range</i>	<i>Storage Size</i>	
byte	-2^7 to $2^7 - 1$ (–128 to 127)	8-bit signed	byte type
short	-2^{15} to $2^{15} - 1$ (–32768 to 32767)	16-bit signed	short type
int	-2^{31} to $2^{31} - 1$ (–2147483648 to 2147483647)	32-bit signed	int type
long	-2^{63} to $2^{63} - 1$ (i.e., –9223372036854775808 to 9223372036854775807)	64-bit signed	long type
float	Negative range: $-3.4028235E + 38$ to $-1.4E - 45$ Positive range: $1.4E - 45$ to $3.4028235E + 38$	32-bit IEEE 754	float type
double	Negative range: $-1.7976931348623157E + 308$ to $-4.9E - 324$ Positive range: $4.9E - 324$ to $1.7976931348623157E + 308$	64-bit IEEE 754	double type

"BIKER BREAK!"



Jan H. Mikkelsen, OOAD11, CoCT, 2026

AGENDA

37

- Introduction to the course (and a bit of public service information)
- Introduction to Object Oriented Programming, as opposed to imperative (procedural) programming
- The Java background
 - The role of Java
 - What is Java actually
- Getting started .. what is needed to start with Java programming
- Example of a simple Java implementation (Hello World)
- Tips & tricks
- Assignments



CODE EXAMPLE

38

- Let us try to revisit the classical "Hello World" example again
- If you did not already do so then install the JDK and copy the following code example into a file called HelloWorld.java

```
1  class HelloWorld {  
2      public static void main(String[] args) {  
3          System.out.println("Hello World!");  
4      }  
5  }
```

- Do not worry about mistakes. The compiler will inform you of any syntax errors and help you fix those – logical errors, on the other hand, are much more difficult to handle



CODE EXAMPLE

- Line 1: Defining the class (class name starts with Capital letter)
 - If public class HelloWorld we have inside/outside package to consider
- Line 2: Defines the main method. This is the entry point for program execution. The 'String[] args' is there to catch any input arguments
- Line 3: Print command System.out.println. Every command must end with ;

```
1  class HelloWorld {  
2      public static void main(String[] args) {  
3          System.out.println("Hello World!");  
4      }  
5  }
```



CODE EXAMPLE

40

- If we want to make use of the 'String[] args' input
 - If we "require" input, then we should check to see if any it there
- The // notation here also indicates comments

```
1  class RepeatTextArgs {  
2      public static void main(String[] args) {  
3  
4          // Check if an argument was given  
5          if (args.length == 0) {  
6              System.out.println("Please provide a text line as an argument.");  
7              return;  
8          }  
9  
10         // Print the text only once  
11         System.out.println(args[0]);  
12     }  
13 }
```



CODE EXAMPLE

41

- Another option is to make use of Scanner (import java.util.Scanner;)
- String is for program launch input while scanner is for interactive input

```
1  import java.util.Scanner;
2
3  class RepeatTextScanner {
4      public static void main(String[] args) {
5          Scanner scanner = new Scanner(System.in);
6
7          // Ask the user for input
8          System.out.print("Enter a line of text: ");
9          String text = scanner.nextLine();
10
11         // Print it only once
12         System.out.println(text);
13
14         scanner.close();
15     }
16 }
```

Imports the Scanner class

Two types of import statements

- Specific import: import a single class as done here
- Wildcard import: import all classes of a package (import java.util.*)



CODE EXAMPLE

42

- Javac is the name of the Java compiler, and we need this as a first step
- IF it is not automatically added to the path you may need to do this your self .. depending on OS
 - Via GUI (system settings)
 - Via command line ..
set PATH=%PATH%;C:\your\path
- As you can see here, Javac can do a wide range of things. Here we focus on the basic functionality
- If interested, you can always look more into this yourself 😊

```
CODE — -bash — 124x31

[(base) AAU138429:CODE jhm$ javac
Usage: javac <options> <source files>
where possible options include:
  @<filename>           Read options and filenames from file
  -Akey[=value]         Options to pass to annotation processors
  --add-modules <module>(<module>)*
                        Root modules to resolve in addition to the initial modules,
                        or all modules on the module path if <module> is ALL-MODULE-PATH.
  --boot-class-path <path>, -bootclasspath <path>
                        Override location of bootstrap class files
  --class-path <path>, -classpath <path>, -cp <path>
                        Specify where to find user class files and annotation processors
  -d <directory>        Specify where to place generated class files
  -deprecation          Output source locations where deprecated APIs are used
  --enable-preview      Enable preview language features.
                        To be used in conjunction with either -source or --release.
  -encoding <encoding>  Specify character encoding used by source files
  -endorseddirs <dirs>  Override location of endorsed standards path
  -extdirs <dirs>       Override location of installed extensions
  -g                   Generate all debugging info
  -g:{lines,vars,source}
                        Generate only some debugging info
  -g:none              Generate no debugging info
  -h <directory>       Specify where to place generated native header files
  --help, -help, -?    Print this help message
  --help-extra, -X     Print help on extra options
  -implicit:{none,class}
                        Specify whether to generate class files for implicitly referenced files
  -J<flag>             Pass <flag> directly to the runtime system
```



CODE EXAMPLE

43

- Once the file is saved we compile it as a first step
- Javac translates our code into something that is not quite machine language but rather a portable hybrid format (byte code)
- After compiling we can see that the output from javac is a .class file of the same name
- The .class file is designed for the JVM and contains everything. The .java file is therefore not needed any longer. The .class file can be executed on all OS

```
CODE — -bash — 124x31

(base) AAU138429:CODE jhm$ javac helloWorld.java
(base) AAU138429:CODE jhm$
(base) AAU138429:CODE jhm$ ls -la
total 32
drwxr-xr-x  5 jhm  staff   160 Jan 31 22:25 .
drwxr-xr-x  5 jhm  staff   160 Jan 31 13:27 ..
-rw-r--r--@ 1 jhm  staff  6148 Jan 31 13:25 .DS_Store
-rw-r--r--  1 jhm  staff   436 Feb  1 10:52 helloWorld.class
-rw-r--r--@ 1 jhm  staff   134 Jan 31 23:19 helloWorld.java
(base) AAU138429:CODE jhm$
```




CODE EXAMPLE

44

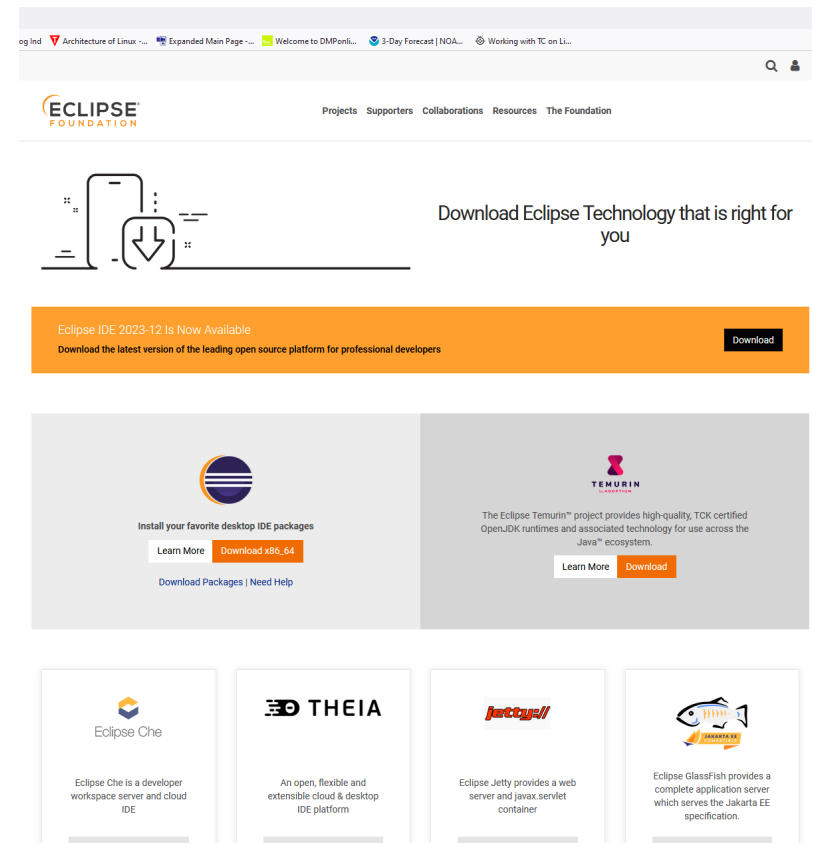
- Command line execution is similarly simple
- Notice that java is case sensitive. This is why the first try is not successful
- In general, the name of the .class file and the same of the source file are identical
- Also notice that java is called without the .class part of the file name

```
CODE — -bash — 124x31
[(base) AAU138429:CODE jhm$ java HelloWorld
Error: Could not find or load main class HelloWorld
Caused by: java.lang.NoClassDefFoundError: HelloWorld (wrong name: helloWorld)
[(base) AAU138429:CODE jhm$
[(base) AAU138429:CODE jhm$ java helloWorld
Hello World
(base) AAU138429:CODE jhm$ █
```

ECLIPSE

45

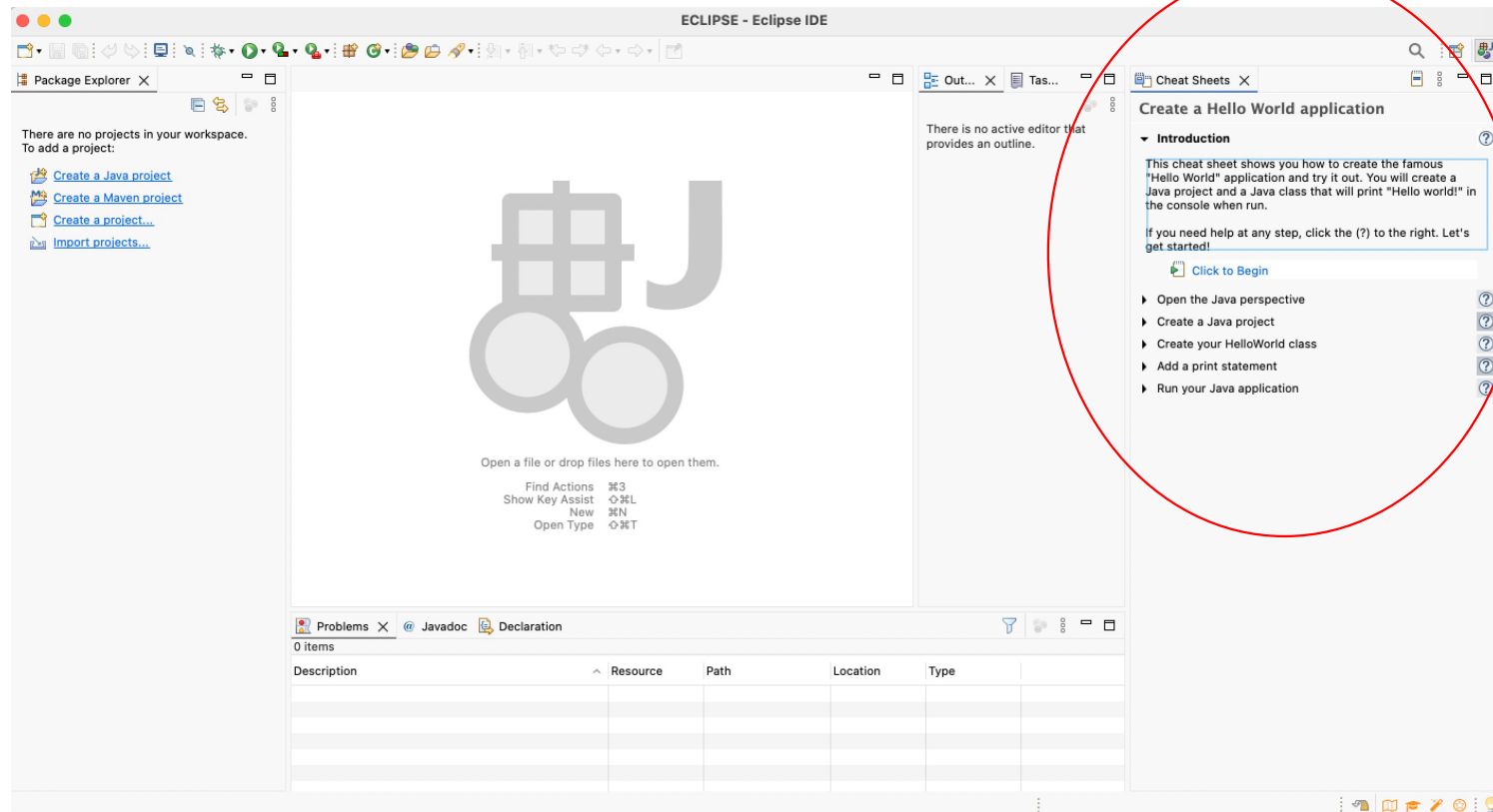
- To get a more “intelligent” environment to work in, an IDE, such as Eclipse, can be a big help
- Download and install Eclipse
<https://www.eclipse.org/downloads/>
- When opening Eclipse for the first time you will have an option to choose a “cheat sheet”, where a Hello World example is implemented
- Please choose this option and try to see if you can manage to follow the steps



ECLIPSE

46

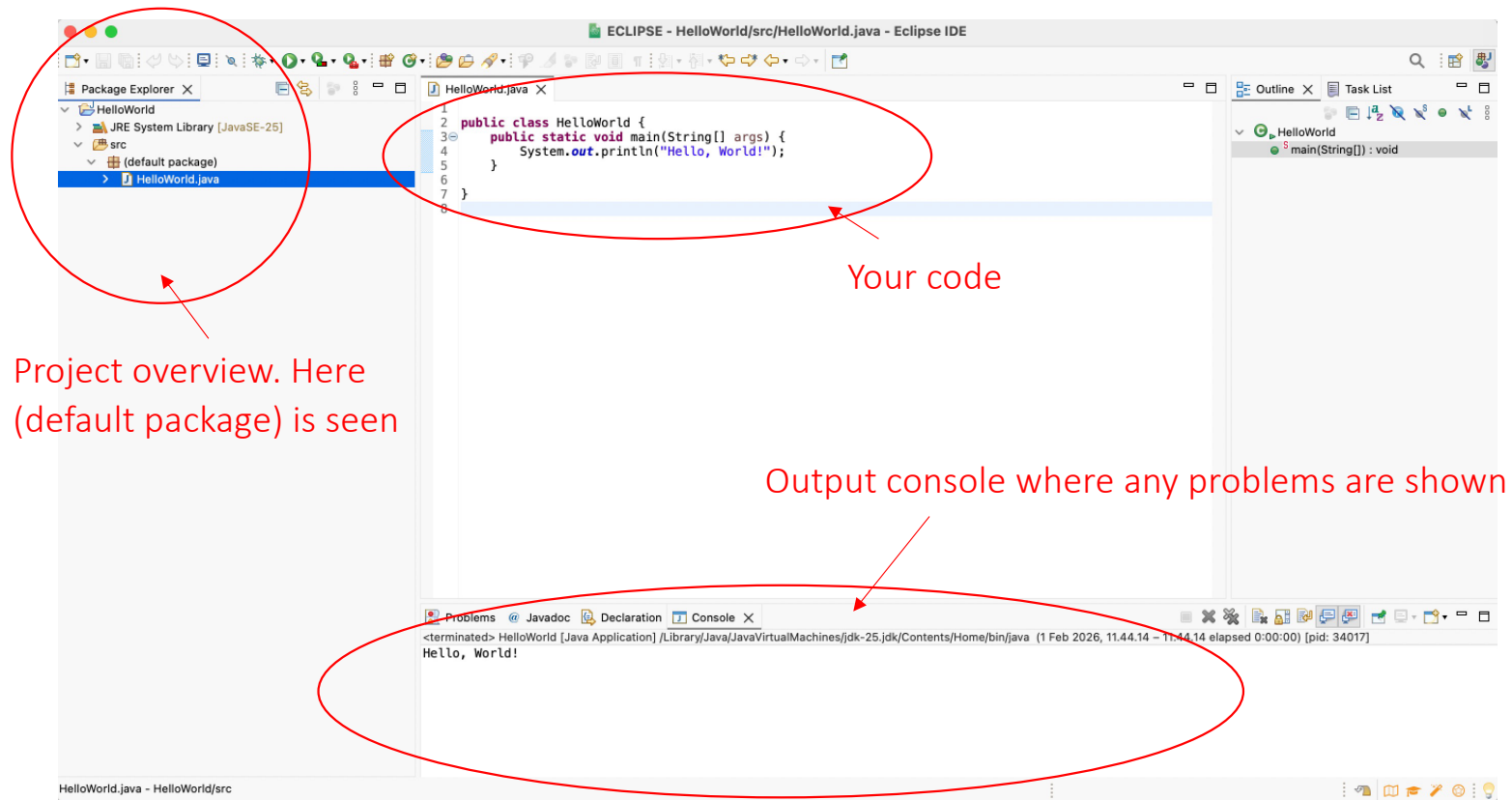
- Having selected "cheat sheet" you should see something like this



ECLIPSE

47

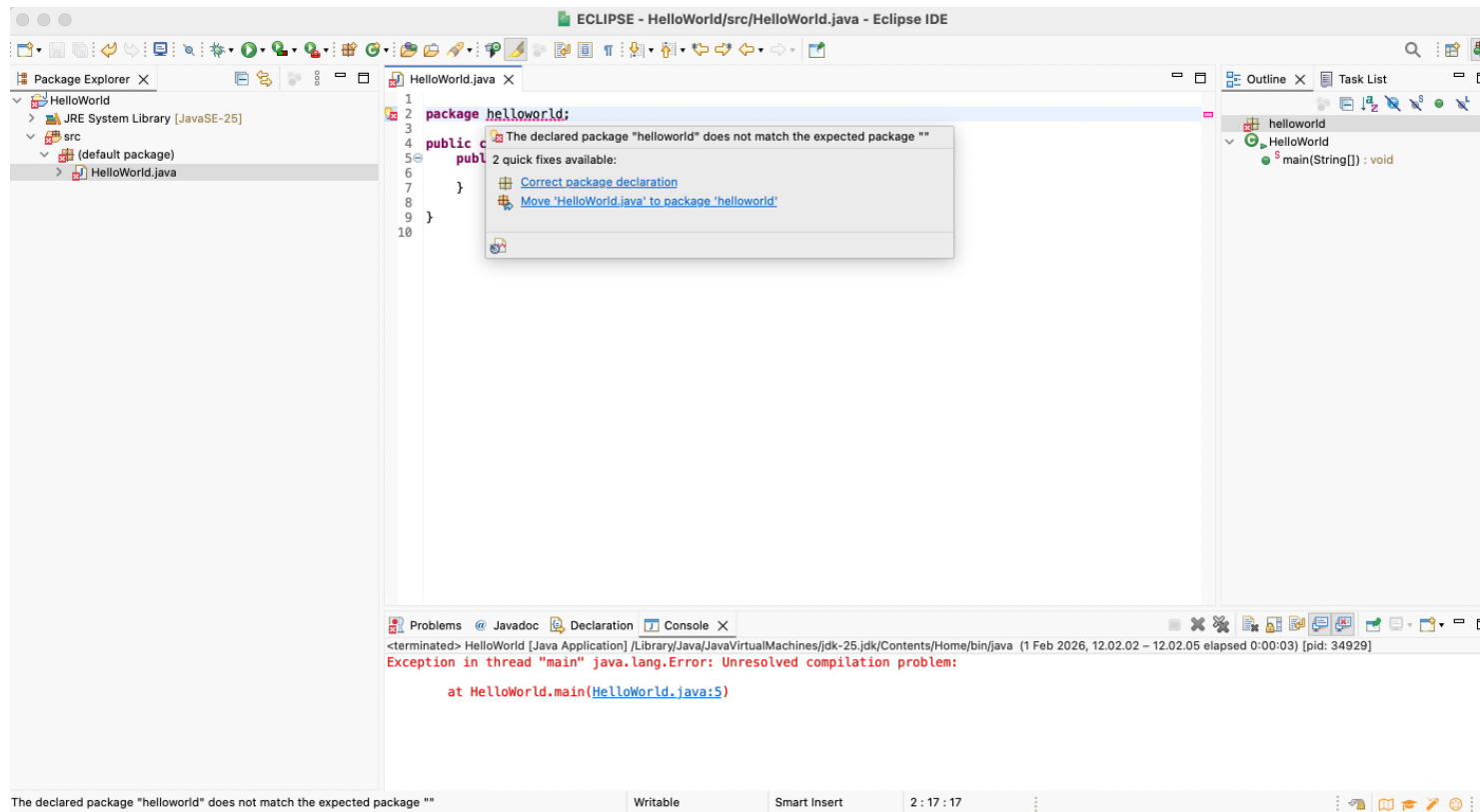
- After going through the steps you should see something like this



ECLIPSE

48

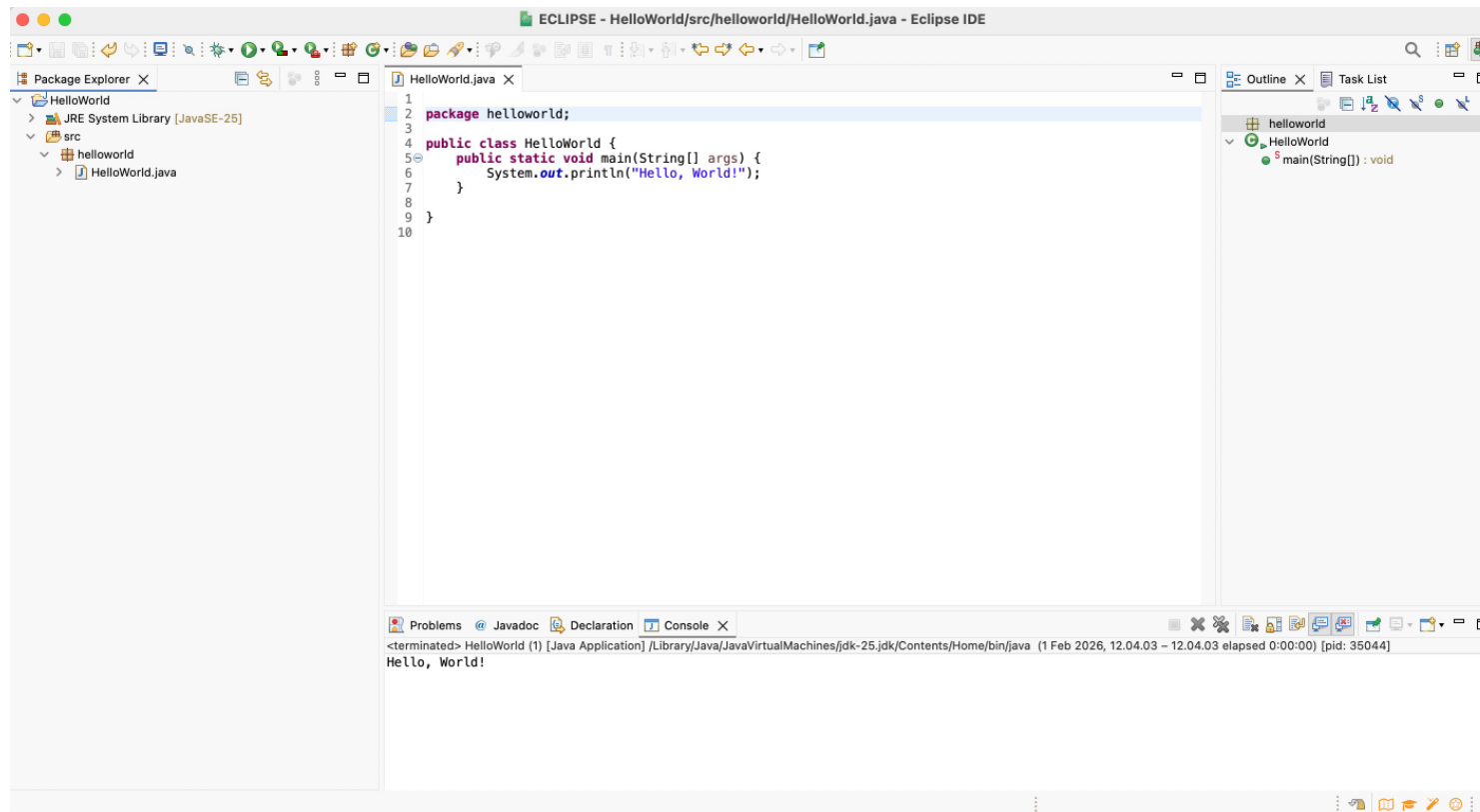
- If we want a bit more structure in our folders we can use package



ECLIPSE

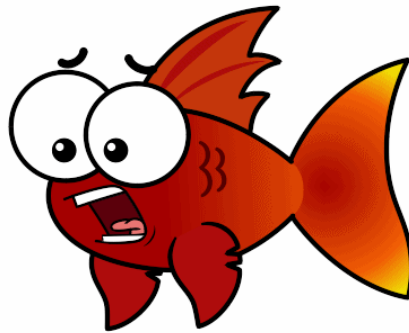
49

- Following this, our code will be located in a folder (a package)



BREAK – PAUSE(FISK)

50



AGENDA

51

- Introduction to the course (and a bit of public service information)
- Introduction to Object Oriented Programming, as opposed to imperativ (procedural) programming
- The Java background
 - The role of Java
 - What is Java actually
- Getting started .. what is needed to start with Java programming
- Example of a simple Java implementation (Hello World)
- Tips & tricks
- Assignments

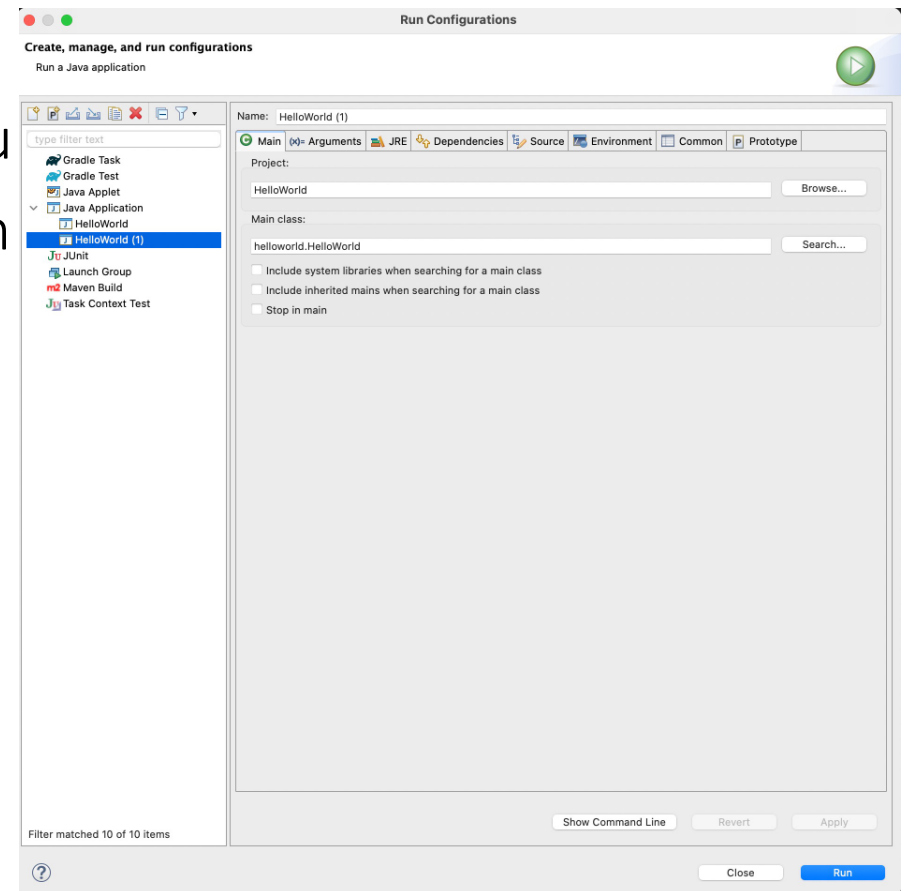
USING PARAMETERS

52

- If we want to execute using code using parameters, we can edit the conditions via the run conditions menu
- The starting point is the default class in the project. Eclipse assumes the starting class to be the most “available” method

```
public static void main(String[] args) {
```

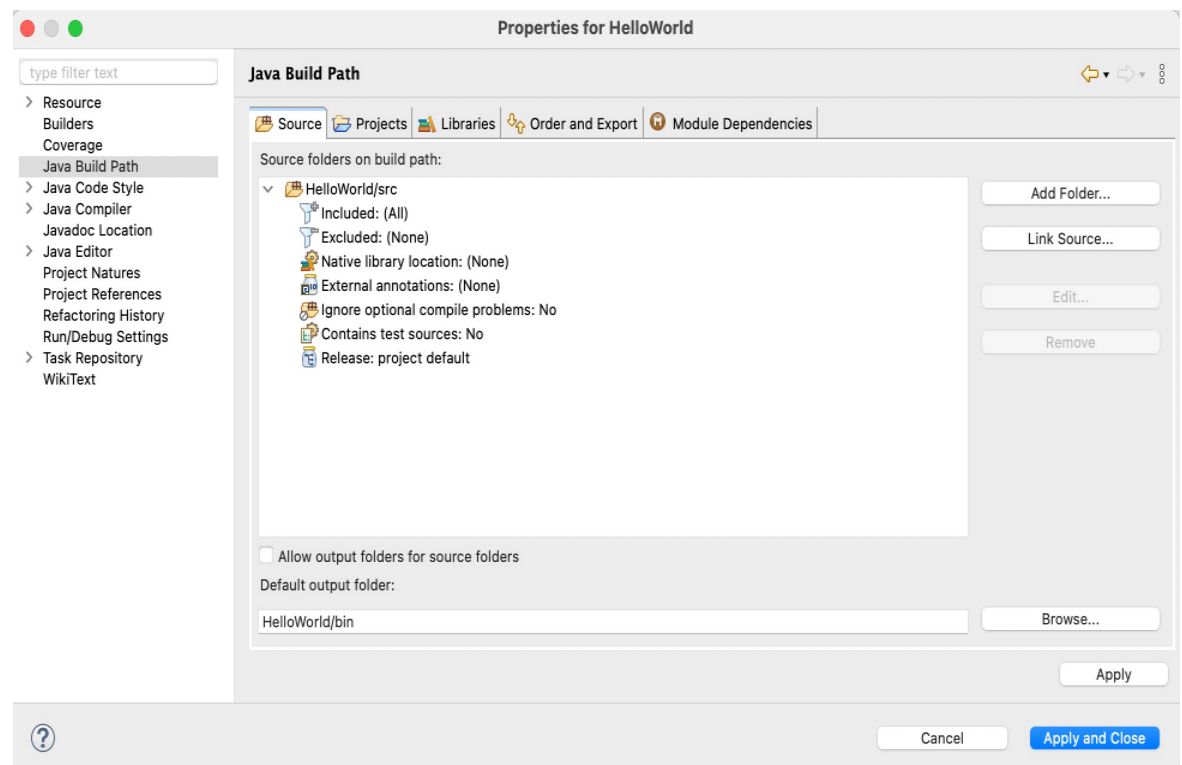
- If your project includes more, it is necessary to choose, since Eclipse will select the first main() it comes across



PROPERTIES FOR THE PROJECT

53

- Often, other libraries than the standard ones are needed
 - This could be JSON libraries, data base drivers etc.
- Build path is important to check as compilation and execution will fail
- The use of command line is also possible but it quickly becomes chaotic and in a practical situation scripts would be needed to handle all of this

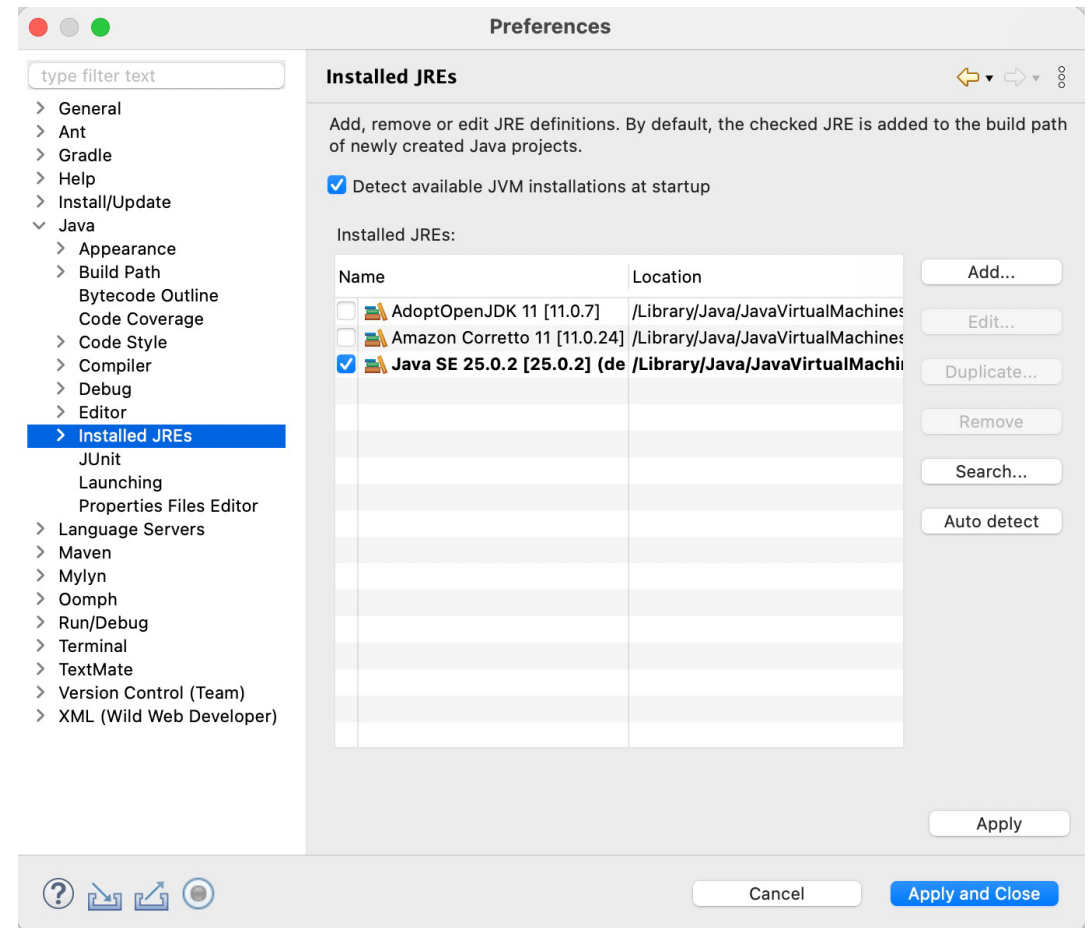
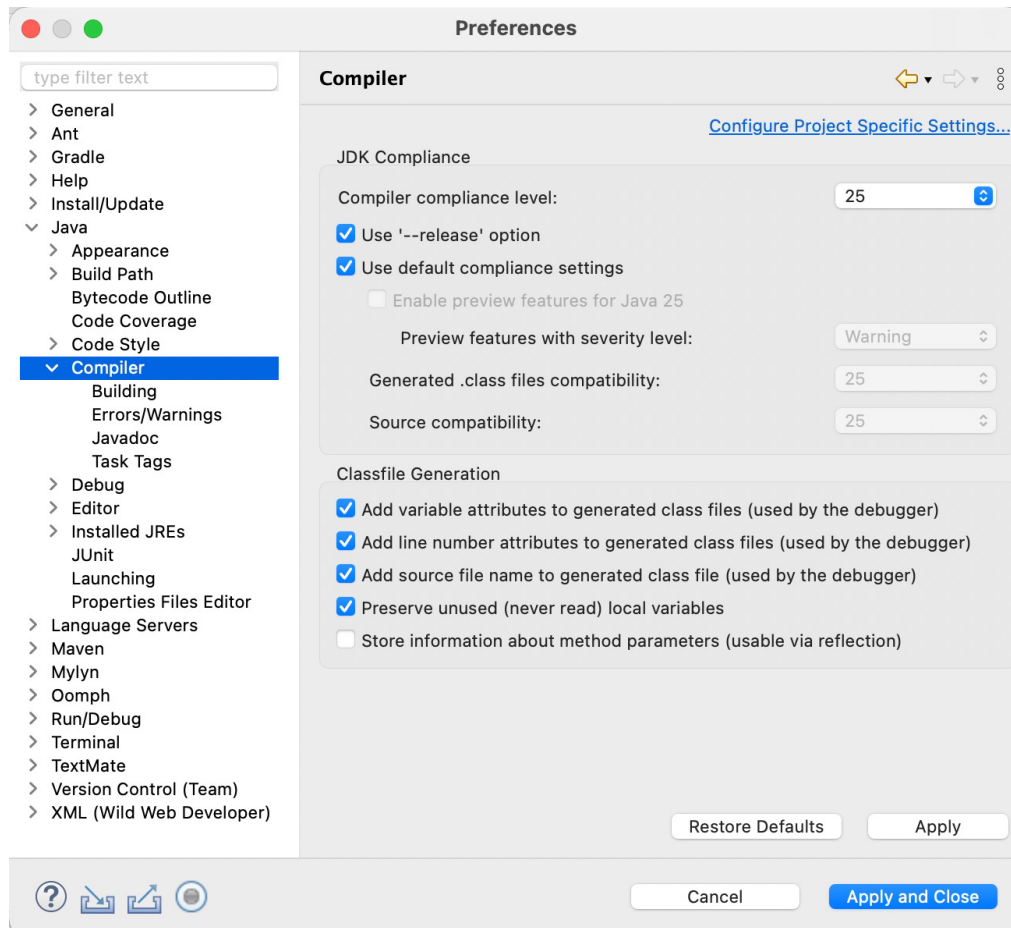


"PITFALLS"

- Eclipse tries to automate as many things as possible for you
- However, sometimes – and for various reasons – things may go wrong
 - Maybe you have several versions of Java installed (unknowingly) Check in compiler settings and make sure to point to the correct version
 - Maybe you have managed to install a JRE and not a JDK? If so, then the compiler part is missing in your setup
 - Despite the strive for compatibility then possibly a specific version of Java is needed
- In addition to the above many other things may go wrong. To help Eclipse offers a long list of (i)relevant setup options – You must try this out a bit for yourself, in case anything is causing issues

"PITFALLS"

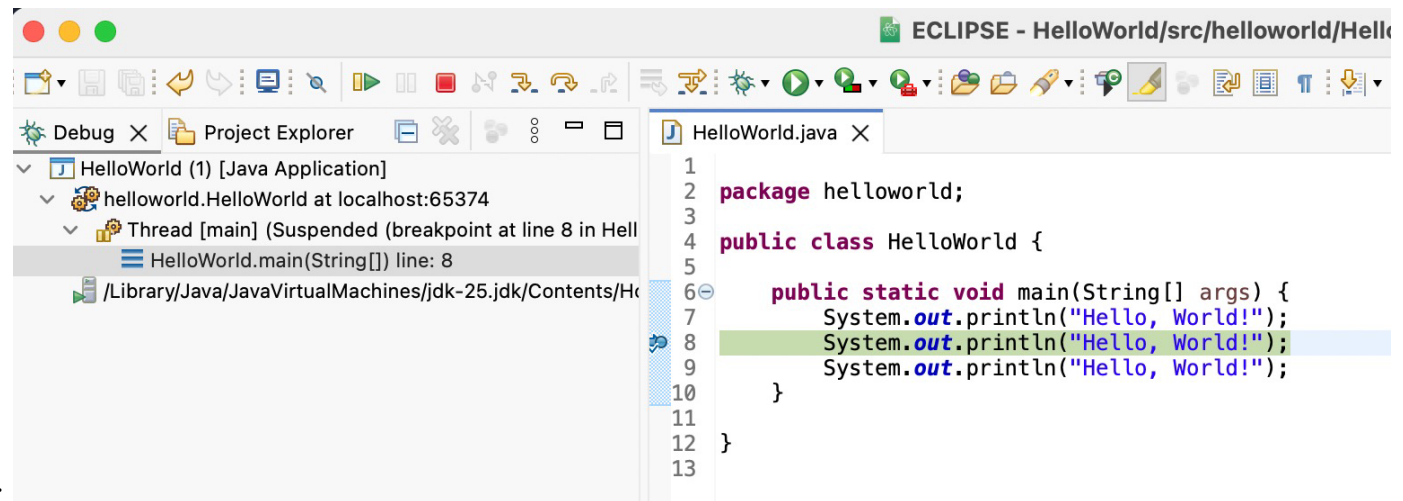
55



USE OF DEBUGGER

56

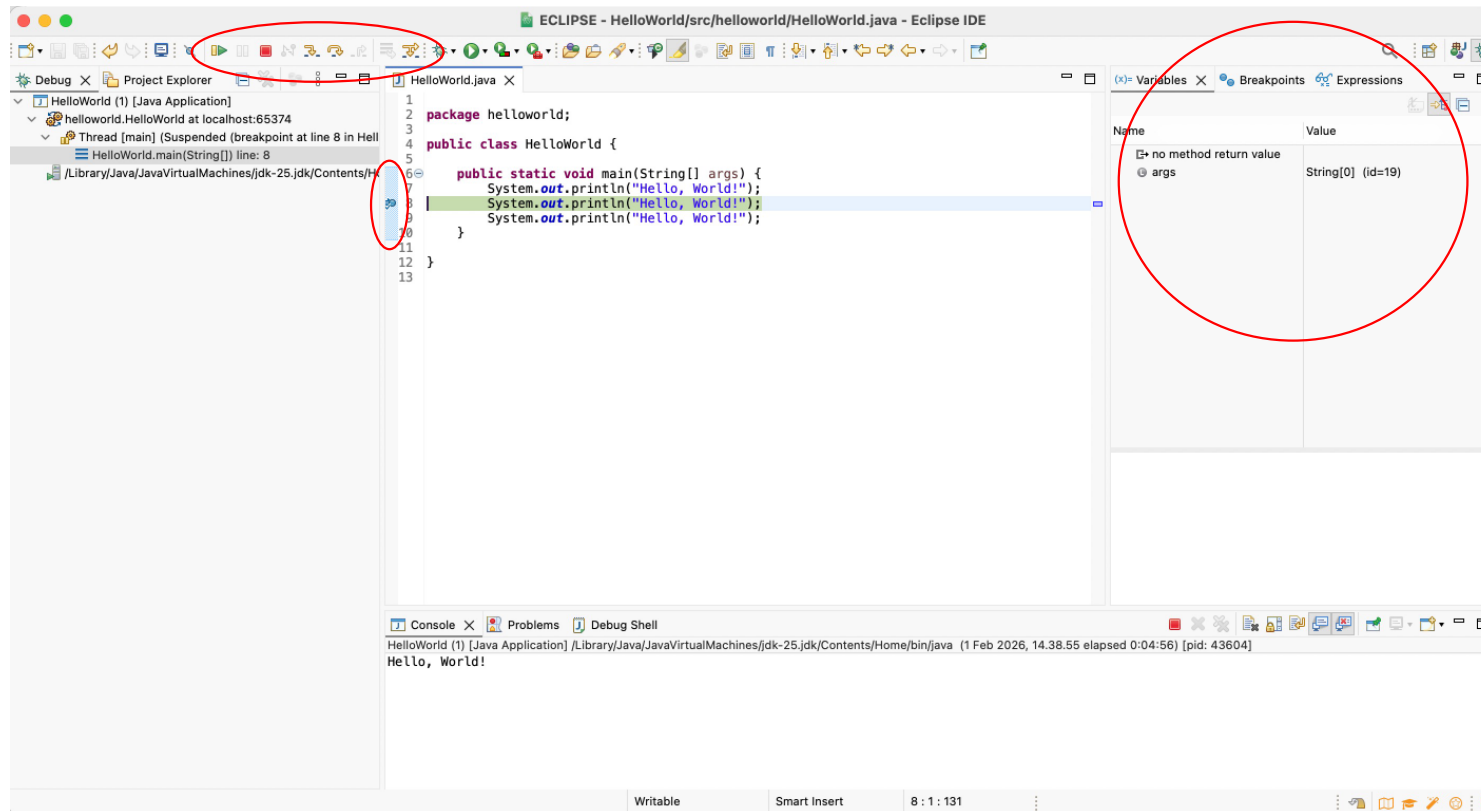
- Marker places where you want the code to stop
- Click the 'bug' to start the debugger
- The code will run to the first setpoint and from here variables, states etc. can be assessed
- Step forward in the code, line by line, or until the next setpoint
- Use your detective skills to determine why things have gone wrong. This may be easy in some cases and in other not at all easy



USE OF DEBUGGER

57

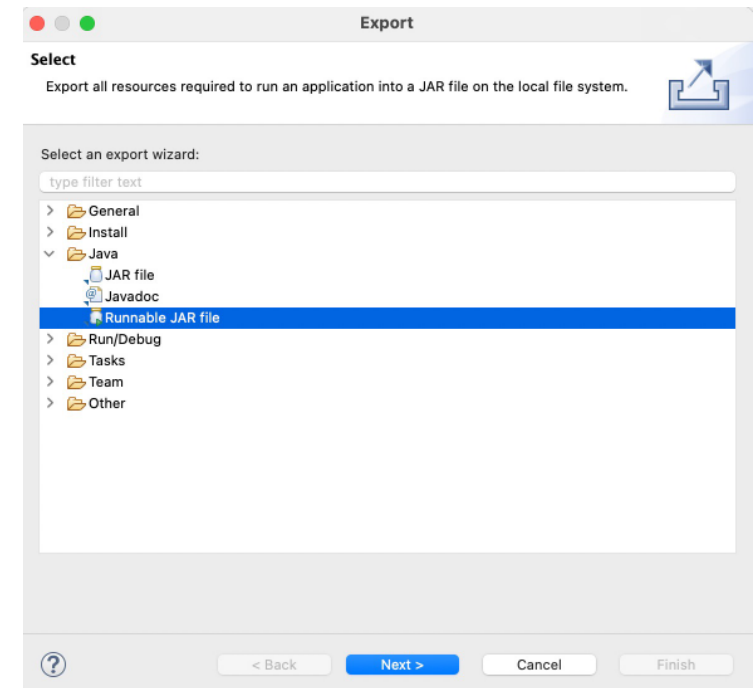
- Here you see an example where the code execution has stopped



EXPORTING A PROJECT

58

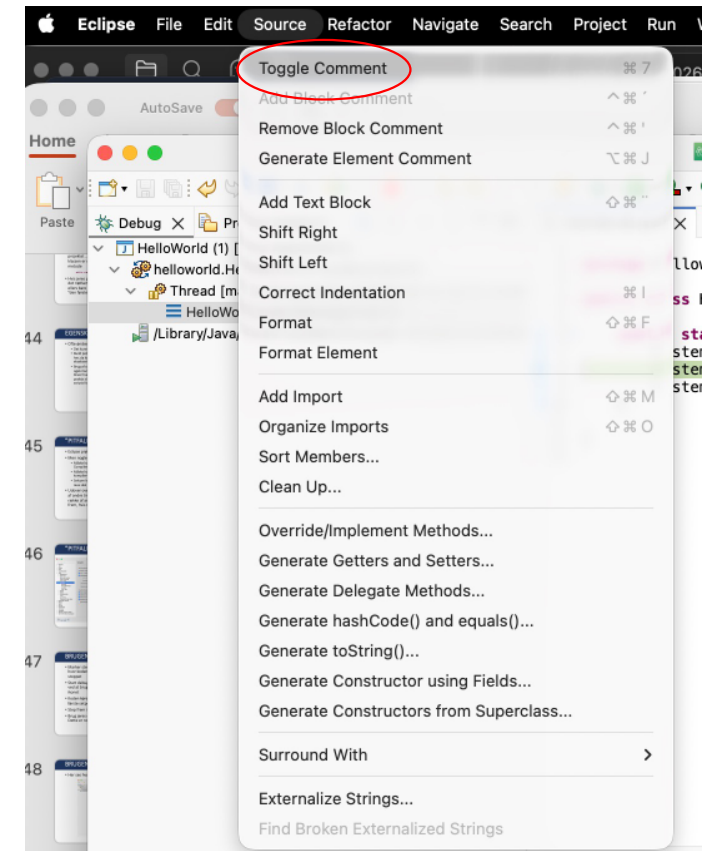
- Sometimes it may be convenient to execute code without the use of the Eclipse environment
- If so, then we can export to a JAR-file
- Collecting everything in an executable JAR-file is a distinctive Java thing
 - To do this we need to define launch conditions
 - Eclipse then packs all required dependencies (well, that is configurable, but by default it does) into a nice little package *filename.jar*
- Then you can run the project directly via the command line
 - `java -jar filename.jar`
 - Some OS even support double-click



OTHER USEFUL FEATURES

59

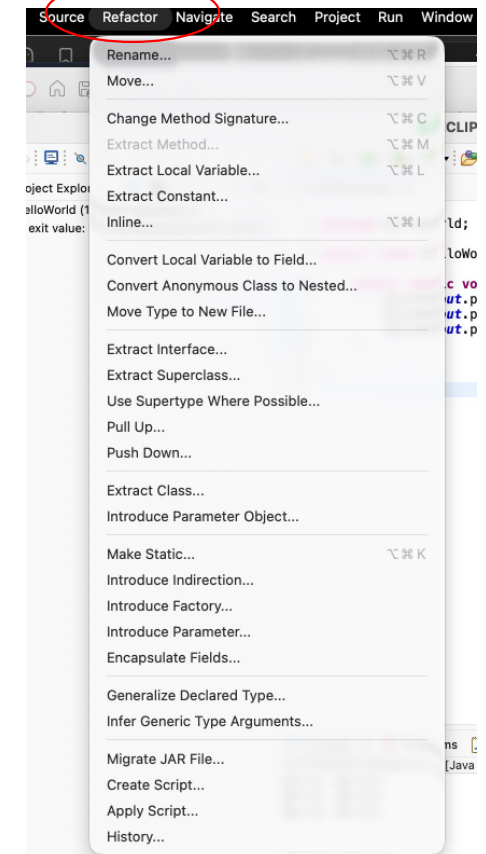
- Toggle comments to automatically include/remove code lines. A nice feature when comparing functionalities
- Correct indentation for nicely looking code
- Automatic method generation
 - Some methods are so standard that most IDEs have automated the code generation for those
 - This increases code-writing efficiency
 - significantly
 - Automatic wrapping of code elements into proper scope {...}



REFACTORING OF CODE

60

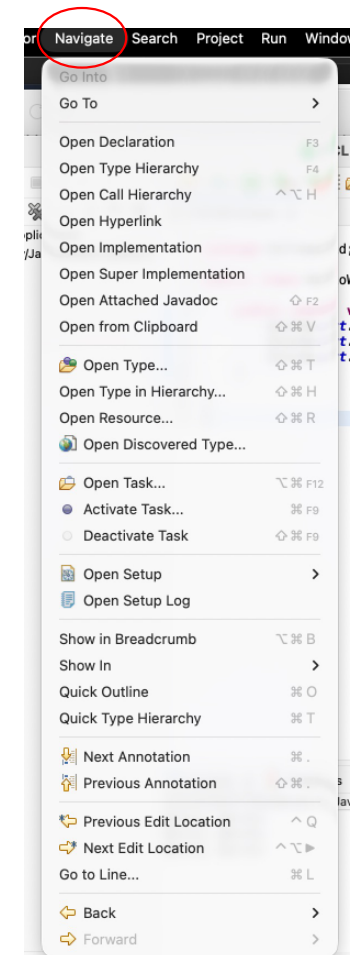
- Once you have a fair amount of code, you may realize that names of classes, methods, attributes etc. should have other names
 - It sucks if you must go through all your +500 files to check if *myClass* should have been *yetAnotherClass* because it fits better to an updated model
- Sometimes code or methods naturally belongs better in other parts, e.g. a parent class
 - Copying/pasting code is generally just a bad idea
- Sometimes you find that your class is better off as an interface – and so forth
- Eclipse is based on years of experience, and can in many cases assist with automation of such tasks



NAVIGATING CODE

61

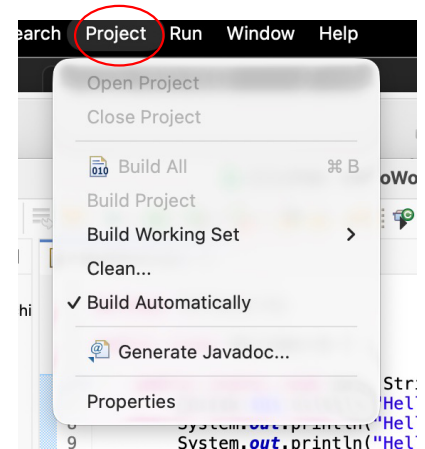
- Particularly for large scale projects, navigating to the right code element may be hard
- Also, who calls who may be highly relevant
 - This may, unfortunately, not be clear from the UML documents leaving us as coders a bit blind
 - So – for your own sake – remember to keep documentation up-to-date!!
- Or you just quickly want to go to the code, where some method is declared
 - This can in larger project be a bit tricky and time consuming
- Most of these functionalities can conveniently also be accessed via a right mouse click



THE PROJECT

62

- Each project may have individual configurations, so you'll need to interact on a per project basis
- Sometimes cleaning up the build can solve some issues
 - In particular, if you have code on a dynamically updated dir, e.g. OneDrive
 - Generate javadoc (code level documentation) can be done by a mouse click
 - Just ensure that you have used commenting effectively in the coder otherwise, your documentation becomes rather boring
- Anything related to e.g. build path, java version etc. is under Properties



THE IMPORTANT STUFF

- Java requires an OO approach to everything but is imperative "under the hood"
- Java has adopted a C/C++ syntax and therefore seems recognisable
- Java is platform independent, but it does require a JRE to achieve this and to handle program execution
- Most things can be handled using a simple text editor and a command line prompt. However, in practice this quickly becomes caotic and an IDE, like Eclipse, can be a life saver
- A (good) IDE, such as Eclipse, is quite complicate and a lot of experience is related to efficient use ... you need to put in the time to harvest the efficiency fruits

AGENDA

64

- Introduction to the course (and a bit of public service information)
- Introduction to Object Oriented Programming, as opposed to imperativ (procedural) programming
- The Java background
 - The role of Java
 - What is Java actually
- Getting started .. what is needed to start with Java programming
- Example of a simple Java implementation (Hello World)
- Tips & tricks
- Assignments



ASSIGNMENTS

65

1. In case you did not already do this, then make the helloWorld example work
2. Write a small program that takes a text line as input and prints this ten times to the consol
3. Write a small program that takes and integer, x, as input and prints a text string of you own choice to the console x times
4. If you used a for-loop structure in 3. then try to rewrite using a while-loop structure and visa versa
5. Implement a program that takes a word (string) as input and then reacts on keywords like for instance “myWord” and writes something else in its place. Define ten words and make the code react to those words (hint: switch)



ASSIGNMENTS

66

6. Using the code on the right it is possible to generate random numbers

```
1 ...  
2 Random rand = new Random();  
3 int rand_int1 = rand.nextInt(100); // random number 0-99  
4 ...
```

Create a guessing-program that takes a number at user input and compares this to a random number. The program should tell the user if his/her number is too high or too low. Give the user either an infinite number of tries to guess the random number or specify a maximum number, say three tries maybe

To run the random number generation code you need to import `java.util.Random` in the beginning of your code